Standalone Progressive Retrieval Attention: A PyTorch Architecture for URI-Addressed Memory Expansion

Jorge Simao
EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 7, 2026

Abstract

This paper describes a standalone PyTorch research prototype for Progressive Retrieval Attention (PRA). A small decoder-only transformer is augmented with explicit reference handles, an in-memory URI resolver, layer-specific chunk memories, one contextual routing gist per chunk, and a cross-attention branch over selected chunk K/V. The corrected implementation routes every batch item independently with a projected last-token query, separates reference and chunk budgets, supports bounded child-first recursive cache construction, and batches variable memory lengths without violating semantic isolation. Summaries are optional routing evidence rather than the primary memory representation. We formalize the reference graph, selection and expansion operators, runtime/cache state, lifecycle, complexity, and training diagnostics. Earlier controlled experiments compared SelfAttention, full PRAttention, and hybrid PRA on WikiText-2 and a fixed reference benchmark. Their ordinary-language and optimization observations remain useful, but their reference-conditioned measurements used a version-1 summary router and whole-reference materialization. A five-seed, eight-step post-refactor smoke matrix now exercises the corrected path at two and five total text splits. It shows reproducible activation of the memory branch, but shuffled-reference loss changes remain below 0.001 on average and do not establish content-causal retrieval. No causal or efficiency superiority is claimed.

1 Introduction

Progressive Retrieval Attention (PRA) is motivated by a common mismatch in long-context language modeling. Many tasks require access to large external context, but the relevant evidence is sparse, hierarchical, and often discovered only after partial reasoning. A conventional decoder-only transformer receives a flat token sequence. Retrieval-augmented generation improves this interface by prepending retrieved passages, but the retrieved text still becomes ordinary prompt context [6]. PRA instead represents external context through lightweight reference handles that can be resolved into URI-addressed memory and attended to through a dedicated memory path.

This paper documents the first standalone PyTorch prototype in the PRA program. The prototype is not a production long-context model. It is a controlled experimental instrument. Its goals are to make the model definition inspectable, expose reference use through explicit metadata, and support ablations that would be difficult to interpret in a large pretrained model. The contribution includes an initial empirical pilot, a three-seed parameter-controlled follow-up, and an evaluation design intended to make failure visible rather than merely demonstrate that a memory branch can lower loss.

The implementation consists of four main components. First, a tiny GPT-style language model provides causal decoder blocks. Second, a reference system parses tokens such as `<REF_1>` and

maps them through an explicit local table to URI-addressed references. Third, a resolver partitions each document into bounded chunks and stores full per-layer token K/V together with exactly one contextual projected-key gist per chunk. Fourth, a PRA attention layer performs hierarchical reference-then-chunk routing and adds a scaled cross-attention branch over only the selected chunk memories.

The easiest way to follow the implementation is as a prepare-then-use pipeline. During preparation, each batch row resolves its own handles, encodes the resulting chunks with the same Transformer stack used for prompts, and records two representations at every PRA layer: a small gist for routing and full token K/V for reading. During use, all prompt rows pass through the Transformer together. The last query in each row ranks only that row's gists; selected chunks are materialized as variable-length memory; and all prompt positions cross-attend to those detailed K/V tensors. Local causal attention never disappears. PRA adds a second, explicitly bounded source of evidence.

2 Model Definition

2.1 Decoder Backbone

Let $x_{1:T}$ be a token sequence. The model embeds tokens and absolute positions:

$$H^{(0)} = E_{\text{tok}}(x_{1:T}) + E_{\text{pos}}(1:T). \tag{1}$$

For each layer $\ell \in \{0, \dots, L-1\}$, the implemented block is a pre-normalized residual decoder block:

$$\tilde{H}^{(\ell)} = \text{LN}_1(H^{(\ell)}), \tag{2}$$
$$A^{(\ell)} = \text{PRAttn}_\ell(\tilde{H}^{(\ell)}), \tag{3}$$
$$U^{(\ell)} = H^{(\ell)} + A^{(\ell)}, \tag{4}$$
$$H^{(\ell+1)} = U^{(\ell)} + \text{FFN}_\ell(\text{LN}_2(U^{(\ell)})). \tag{5}$$

The feed-forward network is a two-layer MLP with GELU activation and configurable hidden width d_{ff}, defaulting to $4d$. The output head maps the final normalized hidden state to vocabulary logits. Configurable d_{ff} permits close parameter matching without changing attention width or depth.

2.2 Causal Self-Attention

For hidden states $X \in \mathbb{R}^{B\times T\times d}$, the local branch computes multi-head causal self-attention:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \tag{6}$$
$$O_{\text{local}} = W_O \,\text{merge}\left[\text{softmax}\left(\frac{QK^\top + M_{\text{causal}}}{\sqrt{d_h}}\right)V\right]. \tag{7}$$

The implementation stores a lower-triangular causal mask with maximum sequence length configured by `PRAConfig.max_seq_len`.

3 Reference Graph and Runtime State

3.1 Reference Graph

Let the external context be a directed attributed graph

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}), \tag{8}$$

where each node $v \in \mathcal{V}$ is a tuple (u, z, s, m, R) containing a stable URI u, content z, optional summary s, metadata m, and an explicit local reference table R. An edge $(v_i, a, v_j) \in \mathcal{E}$ exists only when $R_i[a] = u_j$; URI-prefix scans do not silently invent children. This local namespace makes recursive dependencies inspectable and prevents unrelated objects with similar URI prefixes from entering the graph.

For input sequence x, the reference table exposes a candidate set $\mathcal{H}(x) \subseteq \mathcal{V}$. A resolver

$$\rho(u, p, t) \to (z, s, R, m, \nu) \tag{9}$$

maps URI u, authorization policy p, and logical time or version t to a resolved object. Including p and t in the interface clarifies two systems requirements that the in-memory prototype currently simplifies: access control and freshness.

3.2 Selection and Expansion Operators

At layer ℓ, document u is partitioned into chunks $c \in \mathcal{C}_u$. Encoding produces full token memory $(K_{u,c,\ell}, V_{u,c,\ell})$ and exactly one routing gist per chunk. The default mean gist reconstructs the projected key heads into d dimensions and pools over chunk tokens:

$$g^K_{u,c,\ell} = \frac{1}{|c|} \sum_{j \in c} \text{mergeheads}(K_{u,c,\ell,j}). \tag{10}$$

Alternatives include last-token, explicit reference-end, and a registered GRU pooler. The routing query is the last projected query reconstructed across heads,

$$q_{t,\ell} = \text{mergeheads}\left(X^{(\ell)} W_Q^{(\ell)}\right)_t. \tag{11}$$

Content scores are $a_{u,c,\ell} = \cos(q_{t,\ell}, g^K_{u,c,\ell})$. Hierarchical search aggregates chunk scores within each reference using maximum, mean, or log-sum-exp, selects at most k_r references, and then selects at most k_c chunks independently inside each retained reference. A threshold is applied without converting the two budgets into one flattened top-k. A global-chunk strategy is available for ablation but still enforces both constraints; reference-first routing is accepted only when an explicit reference-level representation is configured. An expansion operator

$$\mathcal{X}(A, \mathcal{G}, b, d) \to A' \tag{12}$$

maps active set A to an expanded set A' under branching budget b and depth budget d. The corrected cache builder implements bounded child-first expansion. It follows only each resolved object's local table, shares reference and token budgets across the traversal, records build states and dependencies, and applies explicit cycle, missing-reference, depth, and overflow policies. This deterministic expansion is a runtime policy, not yet a learned iterative controller.

3.3 Cache and Lifecycle State

The runtime state before token t is

$$\Omega_t = (\mathcal{G}, \mathcal{H}_t, A_t, \mathcal{C}_t, p_t, \nu_t), \tag{13}$$

where A_t is the active reference set, $\mathcal{C}_t$ the resident encoded cache, p_t policy state, and ν_t provenance/version state. A cache entry is keyed conceptually by $(u, v, \ell, m, \theta, p)$: URI, content version, layer, model identity, relevant encoder parameters, and policy domain. The prototype key is only URI because each cache is rebuilt per example; persistent or shared caches require the fuller identity.

One runtime transition can be written

$$A_t = \mathcal{S}_{k,\tau}(q_t, \mathcal{C}_t), \tag{14}$$
$$A'_t = \mathcal{X}(A_t, \mathcal{G}, b, d), \tag{15}$$
$$\mathcal{C}'_t = \mathrm{admit}(\mathrm{encode}(\rho(A'_t))), \tag{16}$$
$$(y_t, \Omega_{t+1}) = \mathrm{attend}(x_{\le t}, \mathcal{C}'_t, \Omega_t). \tag{17}$$

In the corrected prototype, resolution and encoding happen before the batched prompt forward. Each row's cache is ephemeral and private, and a batch-aware wrapper preserves those namespaces while the prompt itself executes once as a tensor of shape $[B, T]$. Recursive construction is available, but admission retains every successfully built entry within fixed budgets and expansion is not learned. Entries move through explicit `unseen`, `building`, `ready`, and failure states so partially built cyclic entries never become searchable. Persistent admission, eviction, and cross-request reuse remain outside the implementation.

4 Reference Handles and Resolver

4.1 Reference Tokens

The prototype uses explicit textual handles of the form `<REF_n>`. A regular expression parser extracts occurrences and resolves them only through the current object's `ReferenceTable`. Each table entry stores an id, token, URI, optional summary, and metadata. Legacy `!!ref:uri!!` syntax is retained only for compatibility.

4.2 URI and Anchor Semantics

References point to simple URIs such as `mem://doc1`. Anchored locations may use URI fragments, for example `mem://doc1#section.a`, but lexical URI ancestry is not semantic graph ancestry. The in-memory resolver stores structured document records containing text, an optional summary, metadata, version, anchors, and a local reference table. A missing URI raises an explicit error; missing-reference policy is owned by the recursive builder rather than hidden inside resolution.

4.3 Resolved References

Given a handle r with URI u, the resolver returns

$$\mathrm{resolve}(u) = (\mathrm{text}_u, \mathrm{summary}_u, R_u, \mathrm{metadata}_u, \mathrm{version}_u). \tag{18}$$

This simple resolver is sufficient for synthetic and local experiments. Future systems can replace it with file, database, vector-index, search, or web resolvers while preserving the same high-level interface.

5 Layer-Specific Memory Cache

5.1 Cache Entries

The PRA cache stores a reference entry containing layer-specific chunk memory:

$$C_u = \left(u, z_u, m_u, \left\{\left(c, K_{u,c,\ell}, V_{u,c,\ell}, g^K_{u,c,\ell}, g^V_{u,c,\ell}\right)\right\}_{c,\ell}\right). \tag{19}$$

Each chunk records a deterministic identifier, source URI, token interval, metadata, full projected token K/V, and one layer-specific routing gist. K/V and gists are detached by default; a trainable-gist mode preserves the registered GRU gist path. Optional summary evidence, when present, is projected contextually into the same layer space and attached to the chunk gist. The implementation exposes an abstract `PRAMemoryCache` and a dictionary-backed `PRASimpleMemoryCache`.

5.2 Reference Encoding Pass

Before a batch forward pass, the training code builds isolated caches from batch metadata. For each referenced document:

1. Resolve the URI to text, optional summary, metadata, and its explicit local reference table.
2. Recursively ensure children are ready first when recursive references are enabled, subject to shared depth, reference, and token budgets.
3. Partition the text with `none`, bounded fixed windows, explicit markers, or a supplied semantic partitioner. Semantic mode without a plugin fails explicitly.
4. Run each document through the same model path. Parent encoding may read already-ready child memory; nonrecursive encoding disables PRA memory.
5. At every PRA layer, project each chunk's token states through that layer's W_K and W_V, then derive exactly one routing gist from projected keys.
6. Store provenance, fingerprints, truncation details, layer-specific chunk tensors, and dependency events before atomically marking the entry ready.

The fourth and fifth steps are the important representation boundary. Reference text is not mapped directly to a single static document vector. A chunk starts at the embedding layer, advances through the model, and exposes the K/V that the corresponding PRA layer would consume. Consequently a lower-layer cache and a higher-layer cache describe the same source at different model depths. The one-vector gist is then pooled from the projected keys at that depth. It is suitable for inexpensive ranking; it does not replace the detailed token memory.

Fixed windows can overlap, but selected materialization deduplicates overlapping source-token spans. Maximum gist counts and overflow policies bound routing state. Cache fingerprints include content, model, tokenizer, and routing identities so a persistent implementation can invalidate stale representations. The current per-example cache remains ephemeral, but the identity rules are explicit.

6 PRA Attention

6.1 Hierarchical Content Routing

At generation or training time, each batch row independently compares its projected last-token query with layer-specific chunk gists. For query $q_{b,t,\ell}$ and chunk gist $g^K_{u,c,\ell}$, the default content score is

$$\text{score}(u, c \mid q) = \frac{q^\top g^K_{u,c,\ell}}{\|q\|_2 \|g^K_{u,c,\ell}\|_2}. \tag{20}$$

This score is a coarse admission decision: it asks which chunks are worth opening for the current row and layer. It is distinct from the token-level attention weights computed after a chunk is opened. The default hierarchical policy uses independent reference and per-reference chunk top-k settings. They are configured as `top_k_references` and `top_k_chunks_per_reference`; either may be zero, yielding no memory. `top_k_refs` remains a warning-producing compatibility alias for one release. Summaries are disabled by default. If enabled, `replace`, `hybrid`, and `augment` policies combine projected summary evidence with content scores; a missing summary always falls back to content routing.

6.2 Memory Cross-Attention Branch

For selected chunk set $\mathcal{A}_{b,\ell}$ and layer ℓ, the implementation materializes only selected full-token memories for each batch row:

$$K^{\text{mem}}_{b,\ell} = \text{concat}_{(u,c)\in\mathcal{A}_{b,\ell}} K_{u,c,\ell}, \quad V^{\text{mem}}_{b,\ell} = \text{concat}_{(u,c)\in\mathcal{A}_{b,\ell}} V_{u,c,\ell}. \tag{21}$$

It then computes a memory branch using the same query tensor Q from the local branch:

$$O_{\text{mem}} = W_{O,\text{mem}} \text{merge}\left[\text{softmax}\left(\frac{Q_b (K^{\text{mem}}_{b,\ell})^\top}{\sqrt{d_h}}\right) V^{\text{mem}}_{b,\ell}\right]. \tag{22}$$

The returned attention output is

$$O = O_{\text{local}} + \alpha O_{\text{mem}}, \tag{23}$$

where α is `memory_alpha`, defaulting to 0.5. If the cache is empty, PRA is disabled, no chunk passes the threshold, or selected memory is unavailable at layer ℓ, the memory residual is exactly zero, including in the presence of output-projection bias.

The normal `selected_chunks` mode transports exactly the token K/V belonging to the routed chunks. Two diagnostic modes make the boundary testable: `full_reference` opens every chunk belonging to a selected URI, while `gist_only` supplies one K/V position per selected chunk. The first removes chunk selection from the experiment; the second asks how much can be done without token-level detail. Neither changes the URI or reference-ranking stage.

Variable memory lengths are executed in one of three modes. Mode zero evaluates each nonempty row independently; mode one pads a single masked rectangle; and mode $K \geq 2$ deterministically partitions rows into at most K contiguous length buckets using either equal-count grouping or a dynamic-programming minimum-padding plan. Masks preserve numerical equivalence, empty rows remain zero, and outputs are restored to original batch order. Diagnostics report actual bucket count, selected versus padded positions, and allocation ratio.

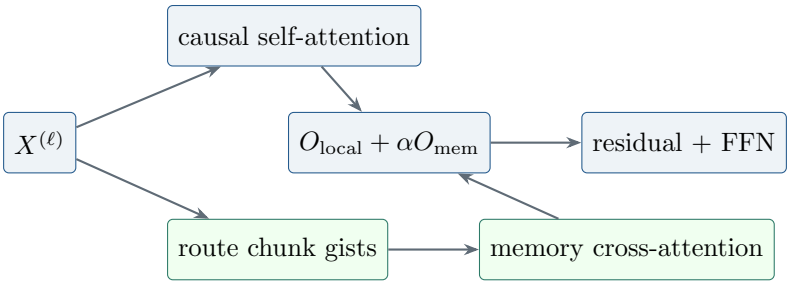

Figure 1: Implemented PRA block. Local causal attention remains intact; selected layer-specific memory contributes through a separately projected residual branch.

6.3 Cross-Attention Rather than Native KV Injection

The standalone prototype uses a separate cross-attention memory branch. Direct concatenation into native self-attention KV is listed as an ablation and future integration path, but it is intentionally not the default mechanism. Cross-attention cleanly separates local causal attention from external memory attention and avoids immediate complications from rotary position embeddings, grouped-query attention, and production cache layout. This makes the prototype suitable for controlled research before adapting large pretrained models.

7 Complexity and Resource Model

Let L be the number of decoder layers, L_p the number of PRA-enabled layers, T local sequence length, r the number of resolved references, m_i the token length of reference i, $M = \sum_{i \in A} m_i$ the selected resident memory length, d hidden width, and w bytes per cached scalar. Ignoring heads and constant factors, local decoder attention costs

$$O(LT^2 d). \tag{24}$$

The prototype separately encodes every resolved reference through the model, costing approximately

$$O\left(Ld \sum_{i=1}^{r} m_i^2\right) + O\left(L_p d^2 \sum_{i=1}^{r} m_i\right) \tag{25}$$

for reference self-attention and layer-specific K/V projection. Memory cross-attention adds

$$O(L_p T M d) \tag{26}$$

to a full-sequence forward pass. The resident K/V payload is approximately

$$B_{\text{cache}} = 2wL_p M d \tag{27}$$

bytes, excluding summaries, object metadata, allocator overhead, and duplicated entries. Each PRA layer adds one $d \times d$ memory output projection plus bias in the current implementation, or $d^2 + d$ parameters. Full PRA therefore adds $L_p(d^2 + d)$ parameters relative to the corresponding custom local block.

These costs explain why shorter visible prompts alone are not a sufficient efficiency result. If every reference is encoded once per example and never reused, $\sum_i m_i^2$ can dominate. PRA becomes

attractive only when selection keeps M small, encoding is amortized across tokens or requests, summaries avoid unnecessary resolution, or quality improves under a fixed compute budget. A systems evaluation must decompose resolver time, tokenization, reference encoding, selection, memory attention, local model compute, cache transfer, and synchronization.

The current batch-aware cache wrapper preserves semantic isolation and permits one $[B, T]$ prompt forward. Reference construction and row-local routing still iterate over logical rows, so they are not yet fully vectorized. A production design would also pack reference encodings and routing candidates with row offsets or masks. Its complexity can approach the same aggregate arithmetic while reducing launch and orchestration cost, provided selection and cache identity remain isolated.

8 Dataset and DataLoader Pipeline

8.1 Dataset Schema

Datasets are represented by three JSONL files per stage:

- `documents.jsonl`: URI-addressed documents, optional summaries, titles, anchors, metadata, and local reference tables.
- `references.jsonl`: reference ids, URIs, summaries, anchors, and metadata.
- `questions.jsonl`: prompts, answers, expected reference ids/URIs, expected chunk ids or spans, and expected anchors.

The loaded sample type is `QuestionSample`, containing a question, answer, list of `ReferenceSample` objects, optional reference/chunk relevance labels, and metadata. Chunk labels are optional: chunk metrics are omitted rather than fabricated when labels are absent. New dataset generators do not emit summaries; loaders retain backward compatibility with legacy optional summary fields and explicit summary experiments. The current registry includes synthetic memory QA, hierarchical synthetic references, code repositories, Wikipedia, books, technical documentation, and GitHub-style repositories.

8.2 Collation

The `PRACollator` tokenizes each question and answer, forms next-token labels, pads variable-length rows, builds a per-sample reference table, and preserves non-tensor metadata. Prompt labels are replaced by the padding/ignore id, so reference-conditioned loss is computed only over answer-continuation tokens. This metadata is essential: the training step uses it to resolve and encode references before the main model forward pass.

8.3 Data Module

The `PRADataModule` used by synthetic and repository-style stages builds a compact tokenizer from questions, answers, summaries, and reference texts. The WikiText-2 study instead trains a 2,000-token BPE tokenizer with whitespace pre-tokenization and reserves `<REF_1>` through `<REF_5>`. The serialized phase-1 tokenizer is restored for phase 2 and for post-fine-tuning plain-text evaluation. This prevents vocabulary drift from masquerading as an architectural effect.

8.4 Reference-Conditioned WikiText-2

The pilot generator retains natural WikiText language and creates *reference-conditioned continuation*, not conventional question answering. Each eligible source entry is divided into one to five earlier parts plus a tail. Earlier parts become URI-addressed references; the final 8–16 words of the tail become the prediction target. The visible prompt contains only reference handles and the neutral instruction `Continue the referenced WikiText passage:`. It therefore does not provide a local lexical prefix from which the target can be copied. Provenance records include the source entry, part boundaries and identifiers, candidate and intended references, tail span, token distance, part count, local-context sufficiency, and relation type.

That first generator labels all earlier parts as intended references and regenerates data from each run seed. We retain its measurements as a version-1 pilot. The controlled follow-up uses a version-2 dataset generated once with seed 1729 and shared across every architecture and model seed. It assigns candidate identifiers and order independently of relevance, marks one adjacent source part as the expected reference, and records source and split provenance. The fixed 512-example corpus yields 409/51/52 train/validation/test examples and a candidate-count-weighted test top-1 chance rate of 0.4202. This removes model-seed-dependent data pairing and permits actual selection metrics, although adjacency remains only a programmatic relevance label rather than human-verified causal evidence.

9 Training Objective and System

The training objective is standard autoregressive cross-entropy:

$$\mathcal{L} = -\sum_{t=1}^{T} \log p_\theta(x_{t+1} \mid x_{\le t}, C), \tag{28}$$

where C is the batch-specific PRA cache constructed from metadata. The training loop uses AdamW, optional learning-rate warmup, gradient accumulation, gradient clipping, optional mixed precision, checkpointing, early stopping, and TensorBoard/W&B/ClearML-compatible logging hooks.

The implemented PRA batch step is:

1. Move tensor fields to the target device.
2. For each example, build a fresh PRA cache from only that example's metadata without attaching it as a global cache.
3. Wrap the completed row caches in a batch-aware cache whose search maps query row b only to cache namespace b.
4. Attach that wrapper once and run one prompt forward on `input_ids` with shape $[B, T]$.
5. Compute cross-entropy with padding ignored.

Per-example cache isolation is semantically important. A previous batch-union implementation allowed each sequence to attend to references belonging to other examples and made within-batch shuffling ineffective. The corrected wrapper keeps the expensive Transformer prompt computation batched without flattening the logical address spaces. Reference-cache construction and routing are still row-wise; those are separate optimization targets from prompt-forward batching.

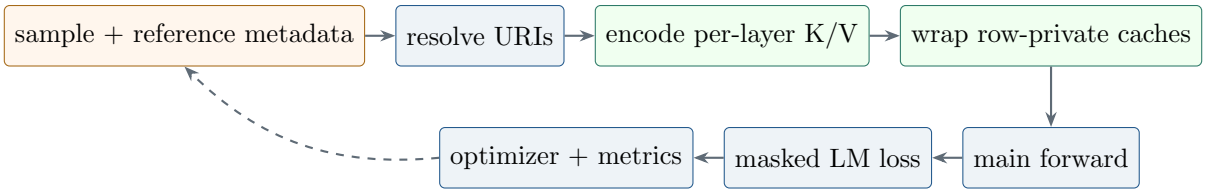

Figure 2: Implemented training/runtime pipeline. Row-private caches are constructed before one batched prompt forward; prompt labels are ignored in reference-conditioned loss.

9.1 Two-Phase Optimization

Phase 1 trains each architecture as an ordinary causal language model with PRA memory absent. The follow-up evaluates three reference-task orders: (i) training from scratch; (ii) restoring the matching phase-1 checkpoint, freezing the base, zero-initializing $W_{O,\text{mem}}$, and optimizing only memory output projections; and (iii) restoring the checkpoint and directly fine-tuning all parameters jointly. Zero initialization makes the initial reference branch an exact no-op. The frozen regime therefore preserves the pretrained local function by construction, while the measured plain-task reevaluation verifies that the implementation honors this constraint. The joint regime tests whether unconstrained adaptation is sufficient and quantifies catastrophic forgetting. The Self-Attention control is invariant to reference ablations and is trained from scratch or jointly as a formatting and optimization control.

9.2 Execution-Time Accounting

The generic training engine records batch, epoch, and complete-run timing. CUDA is synchronized immediately before and after timed GPU regions so queued kernels are included. Reports contain training, validation, test, and wall-clock durations; processed tokens; sequences seen; optimizer steps; optimizer steps per second; and training tokens per second. Training duration sums synchronized batch regions, validation duration sums explicit validation passes, and wall-clock duration includes orchestration and test evaluation. These definitions should not be interchanged when comparing runtime cost.

10 Evaluation Metrics

The evaluation code reports both language-model and reference-behavior metrics:

- **Loss and perplexity**: autoregressive language-model quality.
- **Answer accuracy**: exact containment of the expected answer in generated output or decoded continuation.
- **Reference ranking**: actual selected-URI hit, recall, precision, F1, MRR, and NDCG at k, globally and per layer.
- **Chunk ranking**: selected-chunk hit, recall, precision, F1, MRR, NDCG, and optional source-span IoU against labeled chunks.
- **Expected anchor hit**: whether expected anchors appear in cached text or selected URI strings.

- **Expanded reference count**: average number of ready and recursively expanded references per example, with budget and failure events.
- **Available versus selected memory**: cache references, cache chunks/tokens, selected chunks, and actually materialized memory tokens.
- **Batching efficiency**: requested/actual bucket count, selected positions, allocated padded positions, and allocation ratio.
- **Full-context token count**: token cost of a full-context baseline.
- **Cache hit ratio**: fraction of examples whose expected references are represented in cache.
- **Latency and throughput**: wall-clock evaluation time, examples per second, and tokens per second.
- **Preservation loss**: plain WikiText-2 loss before and after reference-format fine-tuning.
- **Reference ablation loss**: answer-token loss with valid, disabled, shuffled, irrelevant, empty, and oracle reference memory.

These metrics are designed to test the PRA hypothesis directly. A useful PRA model should not merely achieve high answer accuracy; it should do so while selecting the right references and using fewer retrieved tokens than a full-context baseline.

The implemented ablation path supports disabled memory, shuffled references, irrelevant references, empty memory, oracle chunks, all selected chunks, full-reference materialization, and gist-only materialization. These are distinct interventions: cache size is not reported as selected-memory use, and a selected URI does not imply that all of its chunks were materialized. A valid-minus-disabled comparison tests whether the memory branch contributes. Valid versus shuffled or irrelevant is stricter because it asks whether the contribution depends on supplied content. Oracle chunks separate chunk routing from transport. The SelfAttention control is invariant across these conditions by construction.

10.1 Implementation Validation

The corrected mechanics are covered by focused tests rather than presented as model-quality evidence. Tests verify exact projected-key mean pooling; layer- and batch-specific routing; independent reference/chunk budgets; deterministic chunk provenance and overflow; explicit failure for unconfigured semantic chunking; optional summary fallback; GRU-gist gradients and checkpoint registration; recursive child-first build order and cycle safety; no-match equality with the local-only path; and dynamic-bucket output/gradient equivalence. A CUDA smoke test with two batch rows, different selected URIs, unequal memory lengths, and two buckets verifies finite forward/backward execution and independent selections. These checks establish implementation invariants only. The post-refactor training smoke study below exercises counterfactual paths, but controlled multi-seed training remains pending.

10.2 Required Systems Diagnostics

Task loss is insufficient to explain a memory system. Table 1 separates signals already emitted by the prototype from publication-grade diagnostics still required.

Table 1: Systems diagnostics and current implementation status.

Diagnostic	Measurement	Status
Reference selection	hit, recall, precision, F1, MRR, NDCG against relevant URIs	implemented globally and by layer
Chunk selection	ranking metrics and source-span IoU	implemented when labels are available
Attention behavior	routing scores, selected provenance, memory norms	structured trace implemented; heatmaps pending
Layer utilization	selected references/chunks/tokens and branch diagnostics	implemented per layer
Cache reuse	hit rate by URI/version and avoided encoding tokens	per-run hit only; persistent reuse pending
Latency	resolve/cache build, routing, materialization, memory attention, train/eval	cache and attention detail available; full resolver decomposition pending
Dynamic batching	selected versus allocated positions and bucket count	implemented with exact mode comparisons
GPU memory	peak allocated bytes and cache payload bytes	current allocation emitted; peak/decomposition pending
Failure cases	wrong URI, stale content, local insufficiency, poisoning, leakage	traces implemented; labeled taxonomy pending

Attention heatmaps should retain URI and token boundaries rather than concatenate memory into an anonymous axis. Layer utilization should distinguish a selected entry from an entry that receives meaningful attention mass. Cache reuse should count avoided encoding work, not merely dictionary hits. Failure reports should preserve prompt, target, candidate references, selected references, content versions, per-layer behavior, and the counterfactual outputs under disabled and shuffled conditions.

11 Experimental Protocol

The pilot compares three same-width architectures:

1. **SelfAttention**: four PyTorch causal SelfAttention layers.
2. **Full PRA**: four local-plus-memory PRAttention layers.
3. **Hybrid PRA**: two lower SelfAttention layers followed by two PRAttention layers.

All use four layers, four heads, width 256, context length 128, dropout 0.1, and the same 2,000-token BPE vocabulary. The version-1 pilot uses seeds 1 and 7, batch size 2, learning rate 5×10^{-4}, 2,048 optimizer steps, and exactly 524,288 processed tokens per run. The controlled follow-up adds seed 21, extends ordinary-language training to 4,096 steps and 1,048,576 tokens, fixes reference data and splitting with seed 1729, and compares 512-step scratch, frozen-reference-path, and joint reference training at learning rate 2×10^{-4}. All runs execute with Python 3.10.11, PyTorch 2.12.1 with CUDA 12.6, and an NVIDIA GeForce GTX 950M.

The same-width parameter counts are 4,216,320 for SelfAttention, 4,479,488 for full PRA, and 4,347,904 for hybrid PRA. Full and hybrid PRA therefore contain 6.2% and 3.1% more parameters than SelfAttention. In phase 2, the PRA trainable counts are 263,168 and 131,584 because only four or two memory output projections are optimized; the SelfAttention control fine-tunes all 4,216,320 parameters. These differences delimit the claims that can be drawn from the pilot.

Table 2: Current pilot hyperparameters. Phase-2 processed tokens vary slightly with generated example lengths.

Setting	Phase 1: plain WikiText-2	Phase 2: references
Tokenizer	shared BPE, vocabulary 2,000	restored phase-1 BPE
Model	4 layers, 4 heads, width 256	matching phase-1 checkpoint
Context/batch	128 / 2	128 / 2
Optimizer budget	2,048 steps; 524,288 tokens	512 steps; mean 46,301.5 tokens
Learning rate	5×10^{-4}	2×10^{-4}
Dropout	0.1	0.1
References	absent	1–5 parts; top-$k = 5$
Threshold/α	inactive	-1.0 / 1.0
Trainable parameters	all	all SA; memory output only for PRA
Seeds	1, 7	1, 7

The parameter-matched SelfAttention controls retain width 256 and increase each four-layer FFN hidden width from 1,024 to 1,152 or 1,088. The resulting models contain 4,478,976 and 4,347,648 parameters, respectively: 512 fewer than full PRA and 256 fewer than hybrid PRA. This isolates most of the attention-branch capacity difference without changing layer count, head count, or representation width.

The evaluated ablations are:

- valid per-example references;
- a cleared and explicitly disabled PRA path;
- references deterministically shuffled across examples;
- plain WikiText-2 reevaluation after phase 2.

The follow-up additionally evaluated irrelevant, empty, and oracle references plus parameter-matched controls. The corrected runtime now implements chunk-level oracle, full-reference, selected-chunk, gist-only, and recursive modes, but none has yet been rerun as a controlled experiment. Native KV injection, learned triggering, distance-stratified analysis, and memory-strength sweeps remain future ablations.

11.1 Publication-Grade Experiment Matrix

The current pilot is the first row of a larger preregistered-style matrix. Table 3 marks required comparisons without assigning unmeasured values.

Table 3: Required experiment matrix after the controlled follow-up. "Pending" means no empirical claim is made in this paper.

Axis	Required comparison	Status
Reproducibility	seeds $\{1, 7, 21, 42, 87\}$ with intervals/effect sizes	seeds 1, 7, 21 complete
Capacity	same-width plus FFN parameter-matched SA controls	complete at tiny scale
Reference causality	valid, disabled, shuffled, irrelevant, empty, oracle chunks	corrected URI-level smoke complete; chunk-level study pending
Cache budget	entries/tokens/bytes; reuse on/off; eviction policies	pending
Memory strength	α sweep including zero and learned gate	$\alpha = 1$ only
Selection	reference/chunk top-k, threshold, oracle chunks, learned router	fixed-k smoke complete; controlled sweep pending
Transport	separate cross-attention versus native K/V injection	cross-attention only
Adaptation	scratch, frozen branch, LoRA, adapters, joint fine-tuning	scratch/frozen/joint complete
Placement	full PRA, upper-layer hybrid, layer-count sweep	full and 2+2 hybrid complete
Difficulty	distance, candidate count, local sufficiency, recursion depth	recursive runtime present; experiments pending

The five-seed comparison should use fixed tokenizer and split hashes, equal processed-token budgets, and paired seeds. Cache and selector sweeps should initialize from the same checkpoint. Oracle references isolate memory transport from selection; irrelevant and empty controls distinguish content use from generic branch activation. Native K/V injection must account for positional encoding and causal semantics rather than simply concatenate tensors. LoRA, adapters, and joint fine-tuning should be matched by trainable parameter count or reported as separate adaptation regimes.

12 Historical Version-1 Results

Tables 4 and 5 report population means and standard deviations over seeds 1 and 7. They are real report artifacts, but the reference runs predate the corrected hierarchy. They routed a single raw summary embedding per whole reference, reused a batch-collapsed query in an early path, and materialized whole-reference K/V. Accordingly, only the no-memory language-model results transfer directly to the current implementation; reference-conditioned values are engineering history, not evidence for corrected PRA.

All three architectures learn an ordinary token-prediction function and finish within 0.024 test-loss units of one another. This supports the narrow claim that a PRA block behaves comparably to the local baseline when no external memory is present at this scale and budget. It does not establish equivalence under scaling, and the models are not parameter matched.

In that implementation, disabling reference memory increased mean loss by 0.1165 for full PRA and 0.0841 for hybrid PRA, while shuffling increased loss by 0.0145 and 0.0122. These deltas

Table 4: Phase-1 WikiText-2 language modeling at a matched 524,288-token budget.

Architecture	Parameters	Test loss	Perplexity	Token acc.	Train s
SelfAttention	4,216,320	$4.7691 \pm .0036$	$117.82 \pm .42$	$.1264 \pm .0006$	94.07 ± 1.43
Full PRA	4,479,488	$4.7457 \pm .0011$	$115.09 \pm .13$	$.1250 \pm .0012$	90.86 ± 1.13
Hybrid PRA	4,347,904	$4.7574 \pm .0017$	$116.44 \pm .19$	$.1271 \pm .0004$	$95.66 \pm .04$

Table 5: Phase-2 reference-conditioned answer-token loss and plain-text retention. Lower is better.

Architecture	Valid	Disabled	Shuffled	Plain Δ	Train s
SelfAttention	$4.7791 \pm .0734$	$4.7791 \pm .0734$	$4.7791 \pm .0734$	+0.1859	$44.82 \pm .45$
Full PRA	$4.6749 \pm .0607$	$4.7914 \pm .0683$	$4.6894 \pm .0563$	0.0000	45.13 ± 2.83
Hybrid PRA	$4.7111 \pm .0356$	$4.7953 \pm .0284$	$4.7233 \pm .0351$	0.0000	$31.14 \pm .02$

show that the old residual branch changed predictions, but cannot establish correct per-example chunk selection under the corrected semantics. They motivate, but do not substitute for, rerunning counterfactuals with the new reference/chunk traces.

Plain-text reevaluation after phase 2 exactly reproduces the phase-1 losses for full and hybrid PRA because frozen base parameters and zero-initialized isolated output projections protect the local path. The SelfAttention control's mean plain loss rises from 4.7691 to 4.9550, a 0.1859 increase corresponding to about a 20% perplexity increase. This demonstrates preservation by the chosen training strategy, not an inherent guarantee of PRA: the control updates all parameters while PRA updates only reference-specific projections.

Phase-1 mean synchronized validation times are 1.07–1.14 seconds and mean wall-clock times are 97.0–102.0 seconds. Phase-2 mean synchronized training times are 31.1–45.1 seconds, with 46,301.5 mean processed prompt/answer tokens and 512 optimizer steps. These historical timings predate the batch-aware prompt-forward path and must not be used as measurements of its speedup. New reports should separate reference-cache construction, row-local routing, the single prompt forward, and bucketed memory attention.

13 Historical Controlled Follow-Up

The follow-up executed 39 CUDA runs: 15 ordinary-language runs (five capacity/architecture groups times three seeds) and 24 fixed-reference runs spanning SelfAttention controls and scratch, frozen-reference-path, and joint training. Table 6 reports the 1,048,576-token ordinary-language comparison. Population standard deviations are over paired seeds 1, 7, and 21. Reference-task tables in this section use the superseded router and are retained to document optimization behavior, not corrected routing efficacy.

Table 6: Three-seed plain WikiText-2 follow-up at a 1,048,576-token budget.

Architecture	Parameters	Test loss
SelfAttention, same width	4,216,320	$4.6378 \pm .0380$
Full PRA	4,479,488	$4.5463 \pm .0270$
Hybrid PRA	4,347,904	$4.6049 \pm .0359$
SelfAttention, matched to full	4,478,976	$4.5706 \pm .0037$
SelfAttention, matched to hybrid	4,347,648	$4.5725 \pm .0108$

Full PRA is 0.0915 loss units better than same-width SelfAttention, but only 0.0243 better than the nearly parameter-matched control. Hybrid PRA is 0.0323 worse than its matched control. The defensible conclusion is compatibility and approximate parity at this scale, not architectural superiority. Doubling full-PRA pretraining from the earlier 524,288-token pilot to 1,048,576 tokens lowers mean test loss from 4.7457 to 4.5463, a 0.1994 improvement. Full PRA therefore benefits from the longer tested schedule and catches up despite slower early learning; this does not identify an optimal pretraining duration.

Table 7: Reference-task training order on the fixed version-2 corpus. Values are three-seed means; lower loss is better.

Architecture	Order	Valid	Disabled	Plain after	Top-1
Full PRA	scratch	6.3545	6.3762	6.5103	.4199
Full PRA	frozen path	4.7052	4.7344	4.5463	.4119
Full PRA	joint	4.9504	4.9354	4.7060	.4103
Hybrid PRA	scratch	6.3715	6.3929	6.5074	.3718
Hybrid PRA	frozen path	4.7932	4.8146	4.6049	.4006
Hybrid PRA	joint	4.9883	4.9796	4.7478	.3878

Pretraining substantially improves final reference-task loss: direct joint training improves over scratch by 1.4041 for full PRA and 1.3833 for hybrid PRA, while frozen-path training improves by 1.6493 and 1.5783. Frozen training exactly preserves the respective 4.5463 and 4.6049 plain losses because the base is frozen. Direct joint tuning instead increases plain loss by 0.1598 for full PRA and 0.1429 for hybrid PRA. Direct joint fine-tuning is therefore not sufficient under this budget: it both forgets and yields worse reference loss than the isolated path. The hybrid does not benefit more from staging than full PRA; its scratch-to-frozen improvement is smaller, and its staged final loss is higher.

Table 8: Counterfactual reference losses after frozen-path training. Three-seed means.

Architecture	Valid	Disabled	Shuffled	Irrelevant	Oracle
Full PRA	4.7052	4.7344	4.7116	4.7140	4.7051
Hybrid PRA	4.7932	4.8146	4.8016	4.8010	4.7914

The frozen memory branch contributes relative to disabling it, by 0.0292 loss for full PRA and 0.0214 for hybrid PRA. Yet shuffled and irrelevant penalties are only 0.0064–0.0088, and oracle content changes loss by at most 0.0018. Actual test top-1 selection remains near or below the 0.4202 candidate-count chance rate. Periodic validation top-1 curves are nearly flat around their

0.4859 validation chance rate rather than showing faster acquisition after pretraining. Pretraining therefore improves language and final task loss but does not improve generalized reference-selection speed in this experiment.

This negative result is consistent with the version-1 implementation: summaries were compared by a fixed embedding-similarity heuristic, top-k selection was discrete, and no auxiliary routing loss trained the selector against relevance labels. The experiment establishes that a zero-initialized memory residual can be adapted while preserving a frozen local path. It does not establish causal use of correct memory content. The corrected implementation removes raw-summary routing, restores per-row queries, makes layer/chunk provenance observable, and supports oracle chunks. The smoke rerun below exercises those mechanics; the next Phase-1 experiment must repeat the fixed benchmark with matched token budgets, longer training, and the same paired-seed discipline before adding a learned routing objective.

14 Post-Refactor Fixed-Split Smoke Study

After separating the generic training engine into `src/common`, we reran the standalone synthetic workflows, plain WikiText-2 smoke notebooks, and a corrected-router fixed-split matrix. The matrix uses 32 WikiText-2 source entries, paired model seeds {1, 7, 21, 42, 87}, a fixed data-generation and split seed of 1729, one shared 1,331-token BPE vocabulary, four layers, width 256, context length 128, batch size 2, and eight optimizer steps. Split count means total text parts including the prediction tail: split 2 exposes one earlier part, while split 5 exposes four. Source entries, tokenizer, optimizer-step budget, and sequence length are shared across the 20 runs. The target span and processed-token count nevertheless change with the partition, so the comparison is a pipeline and scaling diagnostic rather than an isolated estimate of split-count causality.

Table 9: Post-refactor CUDA smoke matrix. Values are population mean ± standard deviation over five paired model seeds. $\Delta_{\text{use}} = L_{\text{disabled}} - L_{\text{valid}}$ and $\Delta_{\text{content}} = L_{\text{shuffled}} - L_{\text{valid}}$.

Architecture	Splits	Tokens	Test loss	Δ_{use}	Δ_{content}	Train s
SelfAttention	2	903.0	7.1844 ± 0.0350	0	0	0.581 ± 0.068
SelfAttention	5	1,168.4	7.0881 ± 0.0646	0	0	0.884 ± 0.085
Full PRA	2	903.0	7.2207 ± 0.0310	0.0189 ± 0.0030	0.0002 ± 0.0006	0.959 ± 0.218
Full PRA	5	1,168.4	7.0357 ± 0.0542	0.0312 ± 0.0048	-0.0006 ± 0.0009	1.443 ± 0.224

The SelfAttention invariance across valid, disabled, and shuffled conditions is a useful control. Every PRA seed has a positive disabled-path gap in both partition regimes, so memory-branch activation is reproducible rather than a single initialization artifact. Correct content is not. Mean shuffled-reference changes are effectively zero, and split-5 reference top-1 is 0.150 ± 0.102, below the 0.25 uniform-candidate rate. Split-2 top-1 is trivially 1.0 because only one candidate exists. Split 5 raises mean synchronized training time by about 52% for both architectures while increasing processed target-bearing tokens by about 29%. PRA is about 63–65% slower than SelfAttention in these runs. Because each run lasts less than two seconds, startup, synchronization, unequal parameter counts, and unequal token totals preclude quality or systems superiority claims. The defensible contribution is narrower: the corrected batched path, counterfactual evaluator, paired-seed aggregation, split metadata, timing counters, notebook outputs, and HTML/JSON reports execute together without regression.

14.1 Reproducibility and Artifact Scope

Resolved per-seed HTML and JSON reports, plots, timing counters, checkpoints, and aggregate reports are generated under `out/reports`. The aggregate computes population mean and standard deviation without discarding individual seed values and records the fixed data-generation and split seed. Active GPU training time is reconstructed from synchronized batch durations. Content hashes, peak allocated memory, matched token and trainable-capacity budgets, longer training, and independent replication remain necessary before treating the result as publication-grade.

15 Relationship to Prior Work

The prototype builds on the transformer attention architecture [9]. It is related to sparse and long-context attention models such as Longformer, BigBird, Reformer, and Routing Transformer [1, 11, 4, 8], but the central question differs: PRA studies whether external context can remain addressable until needed rather than being placed in a single long sequence.

It also relates to memory-augmented and retrieval-conditioned models. Compressive Transformers maintain compressed memories over earlier activations [7]. Memorizing Transformers retrieve nearest-neighbor memories [10]. RETRO conditions generation on retrieved chunks from a large database [2]. RAG and REALM connect generation to non-parametric retrieval [6, 3]. PRA differs in its emphasis on explicit handles, URI-addressed references, recursive anchors, and per-layer cache construction.

Finally, the implementation anticipates runtime concerns raised by serving systems such as vLLM's PagedAttention [5]. A mature PRA system would require cache admission, sharing, eviction, batching, and memory safety policies.

16 Safety and Systems Correctness

Reference memory expands the model's attack and failure surface. A malicious or compromised reference can poison summaries, embed adversarial instructions, or exploit trusted URI namespaces. URI resolution must therefore preserve source, content digest, version, authorization decision, and resolver identity. Entries from different trust domains should not be merged into an anonymous K/V pool, and outputs should retain enough provenance to reconstruct which objects were resident.

Freshness is part of correctness. A cache key based only on URI can return stale memory after content or model parameters change. Persistent implementations should bind entries to content version, model/adapter identity, layer, representation format, and policy domain. Revocation must invalidate both object lookup and already encoded resident representations.

Batching creates a confidentiality boundary. The corrected prototype constructs a cache per example after a batch-union design exposed cross-example references, then places those caches behind a row-indexed wrapper for one prompt forward. Optimized kernels must preserve equivalent masks and should include adversarial tests proving that logits for one sequence are invariant to another sequence's private cache. Cache reuse across users or tenants requires authorization-aware deduplication.

Recursive expansion can amplify work. The standalone builder now bounds depth, child count, total references, and encoded tokens; detects cycles; exposes missing-reference and overflow policies; and prevents partial entries from becoming searchable. It also fingerprints content, model, tokenizer, and routing configuration. Authorization, latency/byte budgets, revocation, cryptographic provenance, and durable cache invalidation remain deployment requirements beyond the

in-memory resolver.

17 Limitations

The current prototype and follow-up have important limitations. The corrected-router smoke matrix now reaches the planned minimum of five paired model seeds, but eight optimizer steps and 32 source entries make it an engineering pilot rather than a benchmark. Parameter matching controls ordinary-language capacity in the historical study, while the smoke matrix does not match trainable parameters across architectures. The fixed version-2 reference dataset marks an adjacent source part as relevant; it does not provide human-verified causal evidence spans, and the model can reduce continuation loss without learning candidate selection. The near-chance selector and weak counterfactual deltas leave open how much of the memory gain is a content-independent adapter effect.

The corrected selector is still heuristic and receives no auxiliary relevance loss. It uses a projected last-token query rather than explicit reference-token positions, and hard top-k selection remains nondifferentiable. Mean/last gists and full token K/V are detached by default; only the optional GRU gist mode supplies a trainable cache-summary path. Recursive construction is deterministic and child-first, not a learned iterative policy. The training adapter now batches the prompt forward, but reference encoding and routing still iterate over row-private caches. The 20-run fixed-split matrix varies model initialization but keeps one generated dataset and split; summary modes, oracle chunks, full-reference transport, recursion, and dynamic bucketing remain unit- and CUDA-smoke-tested rather than evaluated in a controlled training study. Finally, one tiny model scale cannot establish scaling or efficiency behavior. No broad experimental superiority is claimed.

18 Conclusion

The standalone PyTorch PRA prototype now provides a more faithful testbed for demand-driven, URI-addressed memory expansion: explicit local reference tables, bounded child-first recursion, deterministic chunking, one contextual projected-key gist per chunk and layer, independent reference/chunk routing budgets, selected-chunk K/V materialization, batch isolation, and structured diagnostics. The historical studies establish ordinary-language compatibility and useful optimization controls, but their reference-conditioned conclusions belong to the superseded version-1 router. The corrected architecture has passed focused CPU tests, a multi-row CUDA forward/backward smoke test, and a five-seed end-to-end fixed-split matrix. It has not passed the causal experiment gate: branch use is reproducible, content use is not. The immediate priority is a matched-token rerun with oracle chunks, full-reference and gist-only transport, stronger content controls, explicit chunk labels, and longer training before introducing a learned router or scaling the model.

A Reference-Memory Forward Pass Pseudocode

```
row_caches = []
for sample in minibatch:
    cache = empty_row_cache()

    def ensure_cached(uri, depth, shared_budget, path):
        if uri in path: apply_cycle_policy(); return
        if cache.state(uri) == READY: return
```

```
        resolved = resolver.resolve(uri)
        cache.mark_building(uri)
        shared_budget.consume_reference(uri)

        # Explicit local table only; children become searchable first.
        for child_uri in resolved.reference_table.values():
            ensure_cached(child_uri, depth + 1,
                          shared_budget, path + [uri])

        chunks = partition(resolved.text, routing_config)
        hidden = embed(resolved.text)
        entry = CacheEntry(uri=uri, chunks=chunks)
        for layer in model.layers:
            if layer.has_pra:
                for chunk in chunks:
                    K, V = layer.project_reference_kv(hidden[chunk.span])
                    gist = pool(projected_keys=K)  # exactly one/chunk
                    entry.add(layer.id, chunk.id, K, V, gist)
            hidden = layer(hidden, use_ready_child_memory=True)

        cache.put_ready(detach_by_policy(entry))

    for handle in sample.references:
        ensure_cached(handle.uri, 0, shared_budget(), [])

    row_caches.append(cache)

batch_cache = BatchCache(row_caches)  # query row i sees row_caches[i]
model.set_pra_cache(batch_cache)
logits = model(input_ids)             # one [B,T] -> [B,T,V] forward
loss = cross_entropy(logits, labels, ignore_index=PAD)
```

The preparation loop is intentional: cache identity is part of the example. It no longer implies one Transformer prompt forward per sample. The batch wrapper preserves row ownership during routing, and the attention implementation executes unequal selected memory lengths in masked buckets. A future vectorized cache builder must preserve the same example-to-cache mapping.

B Simplified PyTorch PRAttention

The following excerpt omits thresholds, score aggregation, summary modes, recursion, dropout, and shape utilities while retaining the corrected separation between routing gists and materialized token memory.

```
class PRAttention(nn.Module):
    def __init__(self, d_model, n_heads, memory_alpha=1.0):
        super().__init__()
        self.n_heads = n_heads
        self.head_dim = d_model // n_heads
```

```
        self.q_proj = nn.Linear(d_model, d_model)
        self.k_proj = nn.Linear(d_model, d_model)
        self.v_proj = nn.Linear(d_model, d_model)
        self.local_out = nn.Linear(d_model, d_model)
        self.memory_out = nn.Linear(d_model, d_model)
        self.memory_alpha = memory_alpha

    def forward(self, x, cache, layer_id):
        q = split_heads(self.q_proj(x), self.n_heads)
        k = split_heads(self.k_proj(x), self.n_heads)
        v = split_heads(self.v_proj(x), self.n_heads)

        local_scores = q @ k.transpose(-2, -1) / sqrt(self.head_dim)
        local_scores = local_scores.masked_fill(causal_mask(x), -inf)
        local = softmax(local_scores, dim=-1) @ v
        local = self.local_out(merge_heads(local))

        # Each row routes independently using the projected last query.
        routing_q = merge_heads(q[:, :, -1:, :]).squeeze(1)
        selected_by_row = cache.hierarchical_search(
            routing_q, layer_id,
            top_k_references=kr,
            top_k_chunks_per_reference=kc,
        )
        memory_k, memory_v = materialize_selected_chunks(
            selected_by_row, layer_id
        )
        memory, has_memory = masked_bucketed_attention(
            q, memory_k, memory_v
        )
        memory = self.memory_out(merge_heads(memory))
        memory = memory * has_memory[:, None, None]
        return local + self.memory_alpha * memory
```

Zero-initializing `memory_out` makes the second term exactly zero at phase-2 initialization. The explicit `has_memory` mask also guarantees an exact zero residual for empty rows even when the output projection has a nonzero bias.

References

[1] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[2] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. M. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, 2022.

[3] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Retrieval augmented language model pre-training. In *International Conference on Machine Learning*, 2020.

[4] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations*, 2020.

[5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023.

[6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.

[7] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.

[8] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. In *Transactions of the Association for Computational Linguistics*, 2021.

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[10] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.

[11] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.